

DISEÑO DEL SISTEMA DE INFORMACIÓN

ERP Comercial Amakha

1. INTRODUCCIÓN

1.1 Propósito

El presente documento describe el diseño lógico y arquitectónico del Sistema de Gestión Integral Comercial, estableciendo la estructura técnica que soportará los requerimientos definidos en la Especificación de Requerimientos de Software (SRS).

El objetivo es definir una base sólida que garantice:

- consistencia de datos
 - trazabilidad operativa
 - mantenibilidad del sistema
 - escalabilidad futura
-

1.2 Alcance del Diseño

Este documento cubre:

- Arquitectura general del sistema
- Modelo de datos conceptual
- Componentes principales
- Flujo de información
- Reglas de negocio implementadas
- Decisiones técnicas

No incluye:

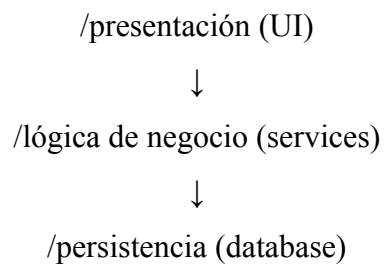
- diseño físico detallado de base de datos (Parte 2)

- diagramas de bajo nivel
 - implementación de código
-

2. ARQUITECTURA DEL SISTEMA

2.1 Enfoque Arquitectónico

Se adopta una arquitectura en capas con separación de responsabilidades:



2.2 Descripción de Capas

Capa de Presentación (UI)

Responsable de la interacción con el usuario:

- formularios
- tablas
- navegación

Tecnología: PySide6

Capa de Lógica de Negocio

Contiene las reglas del sistema:

- validaciones
- cálculos

- control de flujo

Ejemplos:

- cálculo de totales
 - validación de stock
 - gestión de pagos parciales
-

Capa de Persistencia

Responsable del acceso a datos:

- consultas SQL
- inserciones
- actualizaciones

Tecnología: SQLite

3. MODELO DE DATOS CONCEPTUAL

3.1 Entidades Principales

Producto

Representa los bienes comercializados.

Atributos:

- código (PK)
- nombre
- costo
- precio
- stock
- categoría

Persona

Representa actores del sistema.

Atributos:

- dni (PK parcial)
 - nombre
 - rol (cliente/proveedor/vendedor)
-

Operación

Representa una transacción comercial.

Atributos:

- id (PK)
 - fecha
 - tipo (venta/compra)
 - total
 - estado
-

Detalle de Operación

Representa los productos involucrados.

Atributos:

- id
- id_operación (FK)
- producto (FK)
- cantidad
- precio

- subtotal
-

Movimiento de Caja

Representa transacciones financieras.

Atributos:

- id
 - tipo (ingreso/egreso)
 - monto
 - método de pago
 - operación asociada
-

Cuenta Corriente

Representa la deuda de clientes.

Atributos:

- id
 - persona
 - tipo (deuda/pago)
 - monto
 - saldo acumulado
-

3.2 Relaciones

- Una operación tiene muchos detalles
- Un producto puede estar en muchos detalles
- Una persona puede tener múltiples operaciones
- Una operación puede generar múltiples movimientos de caja
- Una persona puede tener múltiples registros en cuenta corriente

4. COMPONENTES DEL SISTEMA

4.1 Módulo de Inventario

- gestión de productos
- actualización de stock

4.2 Módulo de Personas

- registro de clientes/proveedores
- identificación en operaciones

4.3 Módulo de Operaciones

- registro de ventas y compras
- carrito de productos
- cálculo de totales

4.4 Módulo de Caja

- registro de ingresos/egresos
- control de efectivo y virtual

4.5 Módulo de Cuenta Corriente

- registro de deuda
- registro de pagos
- cálculo de saldo

5. FLUJO DE INFORMACIÓN

5.1 Flujo de Venta

Usuario → Operación → Detalle → Stock ↓
→ Caja
→ Cuenta Corriente (si aplica)

5.2 Flujo de Compra

Usuario → Operación → Detalle → Stock ↑
→ Caja (egreso)

5.3 Flujo de Pago Parcial

Pago → Caja → Cuenta Corriente → Actualización de saldo

6. REGLAS DE NEGOCIO IMPLEMENTADAS

- RN01: No se permite venta sin stock disponible
 - RN02: Toda operación debe tener al menos un ítem
 - RN03: Una venta puede generar deuda
 - RN04: Los pagos reducen la deuda existente
 - RN05: Toda operación puede generar movimiento de caja
 - RN06: El stock se actualiza automáticamente
 - RN07: La caja refleja únicamente movimientos confirmados
-

7. DECISIONES TÉCNICAS

7.1 Uso de SQLite

Se selecciona SQLite por:

- simplicidad de implementación
 - bajo costo operativo
 - adecuado para entorno monousuario
-

7.2 Aplicación de Escritorio

Se prioriza:

- independencia de conexión a internet
 - facilidad de uso para usuario no técnico
 - menor complejidad de despliegue
-

7.3 Separación Lógica

Se define una estructura modular para:

- facilitar mantenimiento
 - permitir evolución futura
 - reducir acoplamiento
-

8. CONSIDERACIONES DE ESCALABILIDAD

El diseño permite futuras extensiones:

- incorporación de multiusuario
- migración a base de datos robusta (PostgreSQL)
- integración con sistemas externos
- implementación de reportes avanzados

9. CONSIDERACIONES FINALES

El presente diseño establece una base estructural coherente con los requerimientos definidos, asegurando alineación entre el modelo conceptual del negocio y su implementación tecnológica.

Se prioriza la consistencia, trazabilidad y simplicidad operativa, garantizando un sistema funcional, mantenible y preparado para evolución futura.
